
Actionable Specificity in LLM Code Repair: A Boundary Condition Discovery

Anonymous Authors¹

Abstract

This paper proposes Actionable Specificity (AS), a framework that decomposes verification signal informativeness into three measurable components: localization (AS_loc), state exposure (AS_state), and causal context (AS_causal). The framework is intended to explain why different feedback types yield different LLM code repair success rates. To test whether AS is measurable from standard verification signals—a necessary precondition for testing predictive power—an existence test was conducted on HumanEval and MBPP (664 problems, 600 signals across 6 conditions). Regex-based extraction achieved only 49.5% coverage, with state information extraction at 2.7%. The root cause is that assertion errors, which constitute 53.4% of failures on these benchmarks, produce minimal traceback output lacking file:line structure. Although extraction did not meet the 95% target, the expected AS ordering ($C1 \geq C2 \geq C3 \geq C4$) held on the extractable subset. This work documents a boundary condition: feedback informativeness research must verify extraction feasibility by error type before operationalizing measurement.

1. Introduction

Errors that are most difficult to repair often provide the least actionable feedback. Assertion errors constitute 53.4% of failures on standard code generation benchmarks, yet their error messages contain near-zero extractable state information. This creates a fundamental tension in LLM-based code repair: the errors

most in need of rich diagnostic feedback are those that provide the least.

Recent advances in self-repair demonstrate that iterative refinement with verification feedback can improve pass rates by 4.9–17.1 percentage points on HumanEval. Runtime traces outperform simple error messages, and type constraints improve functional correctness. However, these findings describe which feedback works better without explaining why. The question remains: what properties of a verification signal enable an LLM to localize bugs and infer correct fixes?

This paper hypothesizes that Actionable Specificity (AS) mediates feedback effectiveness. AS is decomposed into three measurable components: localization (AS_loc), indicating whether file:line information is present; state exposure (AS_state), counting how many variable values are visible; and causal context (AS_causal), measuring execution trace depth between assignment and failure. Under this framework, a full runtime trace (high AS) should outperform a bare assertion error (low AS) because it provides localization, state, and causal information necessary for targeted repair.

To test whether AS is measurable from standard verification signals—a necessary precondition for testing its predictive power—an existence test was designed on HumanEval and MBPP (664 problems). 600 signals were generated across 6 conditions (C1–C6) varying in expected AS level, and AS components were extracted using regex patterns designed for Python tracebacks.

The key finding is negative: regex-based extraction achieves only 49.5% coverage, with AS_state at 2.7%. The root cause is that assertion errors, which dominate these benchmarks (53.4%), produce minimal output without traceback frames. Standard AssertionError messages lack the file:line and variable context that extraction patterns require.

This limitation of the operationalization is acknowledged. However, the failure is informative: it reveals a boundary condition that future work must address. The contributions are:

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. The AS framework provides a principled decomposition for analyzing feedback informativeness.
2. Error type stratification is identified as a necessary precondition: feedback informativeness research must verify extraction feasibility by error type before operationalizing.
3. AS ordering ($C1 \geq C2 \geq C3 \geq C4$) is preserved on the extractable subset, suggesting the conceptual decomposition may remain valid under alternative operationalizations.
4. The framework provides actionable guidance by identifying which error types block extraction and why.

2. Related Work

2.0.1. 2.1 SELF-REPAIR AND ITERATIVE REFINEMENT

LLM-based self-repair has emerged as a paradigm for improving code generation accuracy. Prior work systematically evaluated self-repair on HumanEval, finding improvements of 4.9–17.1 percentage points over single-shot generation. Repair success varies by error type, with assertion errors among the more challenging categories. Multi-agent collaborative approaches have achieved 77.13% Pass@1, representing current state-of-the-art performance. However, these works focus on whether repair works rather than why certain feedback enables better repair.

Debugging decay—substantial capability loss after multiple repair attempts—has been identified, suggesting that feedback quality in early attempts is crucial. This motivates a focus on single-attempt success and understanding what makes initial feedback effective.

2.0.2. 2.2 TRACE-BASED DEBUGGING

Prior work has demonstrated that runtime traces outperform error messages for code repair, using debugging-style execution to provide richer context. Self-Debug introduced execution trace methodology for self-repair. While these works establish the superiority of traces over messages, they do not decompose which aspects of traces drive improvement. The AS framework attempts this decomposition.

2.0.3. 2.3 TYPE-GUIDED GENERATION

Type system internalization has been shown to improve functional correctness by providing structured constraints. This suggests that specific, actionable information (types) helps LLMs generate correct code.

The AS framework generalizes this insight: type information contributes to AS_loc (type error locations) and AS_state (type constraints as state).

2.0.4. 2.4 FEEDBACK INFORMATIVENESS GAP

Prior work treats feedback signals as categorical (trace vs. message vs. static analysis) rather than decomposing their information content. Combined compiler and symbolic execution feedback has been used, but without measuring which components drive improvement. Agentic approaches have achieved 42.3% solve rate with 11.8 average iterations, demonstrating that iteration quantity alone is insufficient.

The present work fills this gap by proposing measurable AS components and testing their extractability. The negative result—that regex extraction fails on assertion-dominated benchmarks—reveals a boundary condition that prior work implicitly assumed away.

3. Methodology

3.0.1. 3.1 ACTIONABLE SPECIFICITY FRAMEWORK

The proposed framework hypothesizes that feedback effectiveness for LLM code repair depends on Actionable Specificity (AS)—the degree to which a verification signal provides information that enables targeted bug localization and fix inference. AS is decomposed into three orthogonal dimensions:

AS_loc (Localization): Whether the signal identifies the failure location. Binary: 1 if file:line information is present, 0 otherwise. Extraction pattern: `File '([^\']+)', line (\textbackslash d+)`.

AS_state (State Exposure): How many variable values are visible at the failure point. Continuous: count of variable-value pairs. Extraction pattern: `(\textbackslash w+)\textbackslash s\textit{t}=\textbackslash s\{(['\']*)?\textbackslash w\textbackslash d\.\\-\\[\\{\}]+(['\']*)?\}`.

AS_causal (Causal Context): Execution trace depth between variable assignment and failure. Continuous: count of traceback frames. Extraction pattern: `^\textbackslash s+File '([^\']+)', line (\textbackslash d+), in (\textbackslash w+)`.

These components are designed to be independently measurable from signal text using regex extraction.

3.0.2. 3.2 SIGNAL GENERATION CONDITIONS

Six signal generation conditions were defined, varying in expected AS level:

Condition	Description	Expected AS Level
C1	Full runtime trace	High (all components)
C2	Truncated trace (last 3 frames)	Medium-High
C3	Value-masked trace	Medium (no AS_state)
C4	Error message only	Low
C5	Static analysis (placeholder)	Variable
C6	Syntax error baseline	Minimal

The conditions are ordered such that $C1 \geq C2 \geq C3 \geq C4$ in expected AS, enabling ordered hypothesis testing.

3.0.3. 3.3 DESIGN RATIONALE

The simplest operationalization (regex extraction) was chosen to establish feasibility bounds before investing in more sophisticated methods (AST analysis, LLM-based extraction). This follows the principle of testing necessary conditions first: if AS components cannot be reliably extracted via simple patterns, the framework requires reformulation before testing predictive power.

3.0.4. 3.4 EXISTENCE TEST DESIGN (H-E1)

Before testing whether AS predicts repair success, the existence hypothesis tested whether AS is measurable:

> **H-E1:** Under standard verification signal generation, AS_loc, AS_state, and AS_causal can be independently computed for $\geq 95\%$ of signals, and AS values show expected ordering across conditions ($C1 \geq C2 \geq C3 \geq C4$).

Success Criteria:

- Primary: Extraction rate $\geq 95\%$ across all signals
- Secondary: AS ordering $C1 \geq C2 \geq C3 \geq C4$ preserved

4. Experimental Setup

4.0.1. 4.1 RESEARCH QUESTIONS

RQ1 (Existence): Can AS components be reliably extracted from standard verification signals?

RQ2 (Ordering): Does the expected AS ordering ($C1 \geq C2 \geq C3 \geq C4$) hold across signal conditions?

RQ3 (Independence): Are AS components sufficiently independent to warrant separate analysis?

4.0.2. 4.2 DATASET

HumanEval (164 problems) and MBPP (500 problems) were used, totaling 664 Python programming problems. From 639 failures (96.2% failure rate), 100 failures were stratified-sampled across error types.

4.0.3. 4.3 SIGNAL GENERATION PROTOCOL

For each sampled failure, 6 signal variants (C1–C6) were generated, producing 600 total signals (100 failures $\times$ 6 conditions).

4.0.4. 4.4 IMPLEMENTATION

AS component extractors were implemented using Python regex patterns:

- AS_loc: `File '([^']+)', line (\textbackslash d+)`
- AS_state: `(\textbackslash w+)\textbackslash s\textit{=}\textbackslash s\{(['']?)?\textbackslash w\textbackslash d\.\-\[\{\}]+\textbackslash s(['']?)?`
with excluded keywords (File, line, Error, Exception, etc.)
- AS_causal: `\textbackslash s+File '([^']+)', line (\textbackslash d+), in (\textbackslash w+)` (multiline mode)

Signal variants were generated by truncating traces (C2), masking variable values (C3), extracting error-only messages (C4), and extracting syntax errors (C6).

4.0.5. 4.5 EVALUATION METRICS

Primary Metric: Overall extraction rate = (signals with AS_loc=1) / (total signals)

Component Rates: Per-component extraction rates (AS_loc, AS_state, AS_causal)

Ordering Verification: Mean AS values per condition

Independence: Pearson correlation between components

5. Results

5.0.1. 5.1 MAIN FINDING: EXTRACTION RATE BELOW TARGET

The existence test (H-E1) did not meet the primary criterion. Overall extraction rate was 49.5%, well below the 95% target.

Component	Extraction Rate
AS_loc	49.5%
AS_state	2.7%
AS_causal	15.5%

The extraction rate varies substantially by component. AS_state extraction is nearly absent (2.7%), indicating that variable values are rarely exposed in standard error output. AS_causal achieves only 15.5%, as many errors lack full traceback frames.

5.0.2. 5.2 ERROR TYPE DISTRIBUTION

Error Type	Count	Percentage
Assertion	341	53.4%
Other	171	26.8%
Type	116	18.2%
Syntax	5	0.8%
Index	4	0.6%
Name	1	0.2%
Timeout	1	0.2%

Assertion errors dominate (53.4%) and produce minimal output without traceback frames, explaining the low extraction rate. These errors typically produce output like `AssertionError` without file:line information in the expected format.

5.0.3. 5.3 AS ORDERING PRESERVED ON EXTRACTABLE SUBSET

Condition	Mean AS_state
C1	0.09
C2	0.09
C3	0.00
C4	0.00

The expected ordering $C1 \geq C2 \geq C3 \geq C4$ is satisfied on the extractable subset. C3 (value-masked) and C4 (error-only) correctly show zero AS_state. Al-

though absolute values are low, the relative ordering is preserved.

5.0.4. 5.4 COMPONENT INDEPENDENCE

Component correlations were computed:

	AS_loc	AS_state	AS_causal
AS_loc	1.00	0.16	0.43
AS_state	0.16	1.00	0.17
AS_causal	0.43	0.17	1.00

All correlations are below 0.5, indicating that AS components are reasonably independent. AS_loc and AS_causal show moderate correlation (0.43), which is expected since both depend on traceback frame presence. AS_state shows low correlation with both other components (0.16 and 0.17), supporting the independence assumption.

5.0.5. 5.5 PER-CONDITION ANALYSIS

Examining the raw results data (600 signals), conditions C1–C3 achieve AS_loc=1 for problems with traceback-rich errors, while C4–C6 consistently show AS_loc=0. This pattern holds across error types, but assertion errors show AS_loc=1 only for C1–C3 with minimal causal and state information.

6. Discussion

6.0.1. 6.1 LIMITATIONS OF THE OPERATIONALIZATION

The regex-based extraction achieved only 49.5% coverage, failing to meet the 95% target. This limits the ability to draw conclusions about AS predictive power. The primary cause is the dominance of assertion errors (53.4%), which produce output like `AssertionError` without file:line information.

6.0.2. 6.2 WHAT CAN BE CONCLUDED

Despite the extraction limitation, several findings are informative:

- Error type stratification is necessary:** Future AS research must separate error types before measuring. Type errors and runtime exceptions produce richer tracebacks than assertion errors.
- AS ordering holds where measurable:** On the extractable subset, $C1 \geq C2 \geq C3 \geq C4$ ordering is preserved. This suggests the conceptual

decomposition may be valid under alternative operationalizations.

1. **The framework identifies the barrier:** AS decomposition reveals why extraction fails (missing file:line structure) and suggests solutions (pytest `--tb=long`, LLM-based extraction).

6.0.3. 6.3 ROOT CAUSE ANALYSIS

The extraction failure stems from a mismatch between regex patterns and assertion error output format:

1. **Pattern assumption:** Extraction patterns expect `File 'X', line Y, in func` format from Python tracebacks.
1. **Assertion reality:** Standard assertion failures produce `AssertionError` or `AssertionError: assert X == Y` without full traceback frames when running inline assertions.
1. **Test execution method:** Direct Python execution rather than pytest subprocess captures less structured error output.

6.0.4. 6.4 ACTIONABLE GUIDANCE FROM THE AS FRAMEWORK

The AS framework identifies:

- **Which errors block extraction:** Assertion errors lacking traceback frames
- **Why they block extraction:** Missing file:line format that extraction patterns require
- **How to address it:** Use `pytest --tb=long` for richer tracebacks, or LLM-based extraction that doesn't rely on regex patterns

6.0.5. 6.5 LIMITATIONS

- **Single operationalization tested:** Only regex extraction was evaluated. AST analysis, LLM-based extraction, or pytest verbose mode were not tested.
- **Single benchmark family:** Results are specific to HumanEval/MBPP. Other benchmarks with different error distributions may show different extraction rates.
- **Python-specific:** Trace mechanisms and regex patterns are Python-specific.
- **Predictive power not tested:** Due to extraction failure, the hypothesis that AS predicts repair success remains untested.

7. Conclusion

This paper introduced the Actionable Specificity (AS) framework to explain why different verification signals yield different repair success rates. The existence test revealed that regex-based AS extraction achieves only 49.5% coverage, with near-zero state extraction (2.7%). The root cause is that assertion errors—which constitute 53.4% of benchmark failures—produce minimal traceback output.

This is acknowledged as a limitation of the operationalization, not evidence that the AS concept is invalid. The ordering preservation ($C1 \geq C2 \geq C3 \geq C4$) on extractable signals suggests the decomposition may remain useful under alternative operationalizations.

Contributions:

1. The AS framework provides a principled decomposition for analyzing feedback informativeness.
2. A necessary precondition is identified: error type stratification before extraction.
3. The framework provides actionable guidance by identifying which error types block extraction and why.

Future Work: Testing AS on traceback-rich error subsets, LLM-based extraction, and `pytest --tb=long` for richer diagnostics would address the extraction limitation identified in this study.

References

- AI, M. Agentic apr: Automated program repair via multi-agent iteration. *Technical Report*, 2025.
- Anonymous. Cotran: Code translation with compiler and symbolic execution feedback. *arXiv preprint arXiv:2306.06755*, 2023.
- Anonymous. Codecor: Multi-agent collaborative code repair. *arXiv preprint arXiv:2501.07811*, 2025a.
- Anonymous. Debugging decay: Capability loss in multi-attempt code repair. *arXiv preprint arXiv:2506.18403*, 2025b.
- Anonymous. Tyflow: Type system internalization for code generation. *arXiv preprint arXiv:2510.10216*, 2025c.
- Anonymous. How many tries does it take? an empirical analysis of self-repair in code generation. *arXiv preprint arXiv:2604.10508*, 2026.

-
- Austin, J. et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Chen, M. et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, X. et al. Self-debug: Teaching llms to debug their own code. *arXiv preprint arXiv:2304.05128*, 2023.
- Kang, S. et al. Debugrepair: Runtime trace debugging for llm code repair. *arXiv preprint arXiv:2604.19305*, 2026.